

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a3218d23fd6025b63.jsonl`
- SHA-256: `6ea87345809ca7fefe58c458001d3b33fdb232e1e70142e6a5742529f38`
- Source modified: `2026-07-02T04:55:36+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T04:45:39.421Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.

REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).

AUDIT FINDINGS YOU ARE FIXING: R1-20 (poll timeout abandons running GPU execution) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.

TASK BRIEF:

In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:
backend/compute/cloud_run.py ~:173 ? on bucket-poll timeout the code abandons a still-running L4 execution (bounded only by --task-timeout 3600). Fix: on PolicyTimeout do ONE final done-marker check, then best-effort cancel via the Jobs API executions.cancel using the stored execution name (lazy SDK import, guarded like run_job's project guard from #342), and include the execution name in the failure message so the operator can verify in GCP. Never mask the original timeout error if cancel fails. Tests with the mocked client: cancel called with right name, cancel-failure still surfaces PolicyTimeout, late-done-marker rescues the result.

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):

1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.

COMMIT/PR CONVENTIONS:

- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.

QUALITY BAR:

- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md \$0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-

Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:47.186Z)

193-[Omitted long context line]
194-[Omitted long context line]
195-
196-### R1-17 [P2/M] tech-debt ? badminton-highlight-indexer
197-**Fetched-input scratch copies are never reclaimed anywhere (process jobs, compile jobs, /api/assets, cloud_run marker reads) ? 'OS temp reclamation' premise is false on Windows**
198-- anchor: `backend/storage/\_\_init\_\_.py:89` · tracked in: ? · finder: engine:correctness
199-[Omitted long context line]
200-- why it matters: Slow-burn disk exhaustion on exactly the appliance the product ships as; also inflates the free-space guard's false rejections.
201-- proposed fix: Wrap remote-input acquisition in a per-job scope that deletes the scratch dir in a finally (worker already owns the job lifecycle), or route scratch under output_base(cfg)/tmp with an age-based sweep at startup; make cloud_run._read_json use TemporaryDirectory.
202-[Omitted long context line]
203-[Omitted long context line]
204-
205-### R1-18 [P2/S] critical-fix ? badminton-highlight-indexer
206-**GDriveApiStorage.fetch_to_local buffers the entire file in RAM (get_media().execute()); put() deletes the old object before the new upload succeeds**
207-- anchor: `backend/storage/gdrive\_api.py:144` · tracked in: ? · finder: engine:correctness
208-- evidence: gdrive_api.py:143-148: `data = self.\_service.files().get\_media(fileId=file\_id).execute()` then `fh.write(data)` ? the whole object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)` BEFORE `files().create(...)` ? delete-then-create is not atomic.
209-- why it matters: Latent OOM + data-loss on the Drive-API storage hot path before it goes live for the reel round-trip.
210-- proposed fix: Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for put(), create the new file first (or upload to a temp name) and delete the old one only after success.
211-[Omitted long context line]
212-[Omitted long context line]
213-
214-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
215-**config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**
216-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness
217-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global\_config.output.output\_dir = resolve\_output\_dir(args.output\_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir overrides this at startup' ? implying only when given). db_path derives from it (main.py:1995-1997).
218-- why it matters: A documented config knob that is a no-op on the main entry path, with data-location consequences (perceived data loss / split state).
219-- proposed fix: Change argparse default to None and only override when args.output_dir is not None (falling back to the config value through resolve_output_dir).
220-[Omitted long context line]
221-[Omitted long context line]
222-
223-### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer
224-**upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged --app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live

```

under RALLY_APP_HOME**
225-- anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness
226-- evidence: uploads.py:50 `upload_dir = getattr(sec, 'upload_dir', None) or
'output/uploads'` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no
resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved,
main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.
227-- why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly
the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).
228-- proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg
(relative ? app-home-rooted; absolute passes through), mirroring output_dir.
229-[Omitted long context line]
230-[Omitted long context line]
231-
232-### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer
233-**Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no
cancellation, so the L4 keeps billing and its late result is never ingested**
234-- anchor: `backend/compute/cloud_run.py:173` · tracked in: ? · finder: engine:correctness
235-- evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...);
execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The
exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ?
no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the
late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).
236-- why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-
gated go-live area, but the code path is live behind compute='cloud').
237-- proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API
(executions.cancel with the stored execution name) and log the execution id in the failure
message; optionally check the done-marker once more before failing.
238-[Omitted long context line]
239-[Omitted long context line]
240-
241-### R1-22 [P2/M] tech-debt ? badminton-highlight-indexer
242-**Stage-2 AI-handoff orchestration duplicated between the default segmenter (yolo_hybrid)
and the trajectory family**
243-- anchor: `backend/pipeline/segmenters/trajectory_hybrid.py:390` · tracked in: ? · finder:
engine:structure
244-[Omitted long context line]
245-- why it matters:
246-- proposed fix: Extract a shared handoff helper (e.g.
backend/pipeline/segmenters/handoff.py: pad_window + process_candidate_window + the executor
fan-out) consumed by both HybridYoloSegmenter and TrajectoryHybridSegmenter; move _fmt_ts to
backend/utils. Note the trajectory subclass family itself (fusion/tracknet/wasb via
_build_runner template methods) is well-factored ? the duplication is only across the
yolo/trajectory family boundary.
247-[Omitted long context line]
248-[Omitted long context line]
249-
250-### R1-23 [P2/L] tech-debt ? badminton-highlight-indexer
251-**Four more god modules over 800 lines; database.py is a 50-method monolith**
252-- anchor: `backend/infrastructure/database.py:25` · tracked in: mpyy.ini comments reference
the typed-config migration (D3) in docs/CODE_CLEANUP_AUDIT_2026-06-24.md · finder:
engine:structure
253-[Omitted long context line]

```

3. user (2026-07-02T04:45:47.418Z)

```

1      # CLAUDE.md ? project memory / agent handoff
2
3      Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4      FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5      repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7      ## Read these first
8      - **docs/README.md** ? doc index. **docs/CODE_MAP.md** ? code navigation + data
9      contracts (the cold-start map; read before touching code).
10     - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+

```

```

the
11     live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12     disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13     - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14     `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15     (offline-segmenter narrative) and `docs/BACKLOG.md`.
16     - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
17     `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18     (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19     "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
20     eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21     (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22     - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23     **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
24     **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25     archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
26     note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27     lifecycle register in **`docs/DOC_STATUS.md`**.
28
29     ## Current state (June 2026)
30     - **Scaffolding complete** ; strategy foundation + owned-model roadmap merged. **No
31     platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32     sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
33     validation, reliability, CI + test coverage) ? see git history.
34     - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35     (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36     greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37     implementation without explicit owner approval.**
38
39     ## Architecture in one breath
40     - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41     / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42     `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
43     - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
44     - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
45     user_version` migration ladder in `backend/infrastructure/database.py`.
46
47     ## Committed guardrails (non-negotiable)
48     - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49     training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50     code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51     - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52     is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel

```

```

53         needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54         minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55         - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56         (GDPR Art. 9).
57         - **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.
58         - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/`;
59         Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60         on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61         DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62         `deploy/digitalocean/` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64         ## Workflow conventions
65         - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66         - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67         - Never push to the default branch directly; descriptive commit messages.
68         - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69         - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
70         - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
71         who has the golden corpus + labelled videos; verify pure-logic paths locally.
72

```

4. user (2026-07-02T04:46:01.137Z)

R1-21 [P2/M] critical-fix ? badminton-highlight-indexer

```

**Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no
cancellation, so the L4 keeps billing and its late result is never ingested**
- anchor: `backend/compute/cloud_run.py:173` · tracked in: ? · finder: engine:correctness
- evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...);
execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The
exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ?
no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the
late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).
- why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-
gated go-live area, but the code path is live behind compute='cloud').
- proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API
(executions.cancel with the stored execution name) and log the execution id in the failure
message; optionally check the done-marker once more before failing.
- verifier notes: Correction to the failure scenario: "the L4 keeps billing" is bounded, not
open-ended ? the job is created with --max-retries 0 --task-timeout 3600
(deploy/cloudrun/README.md:33), so the orphaned execution bills at most ~30 extra minutes past
the 1800s poll deadline, then Cloud Run kills it. The sharper, systematic scenario is the
policy/task-timeout mismatch (1800s poll vs 3600s task budget): a long match video whose GPU
inference takes 30-60 min ALWAYS ends as a local "cloud dispatch error" failure while the cloud
run later succeeds, writes the done-marker + outputs to the bucket, and the result is silently
wasted ? user pays full GPU cost and must re-dispatch. Fix scoping corrections: (a) cancellation
must be a new abstract method on CloudRunJobClient (the injected test seam) with the real impl

```

using `run_v2.ExecutionsClient().cancel_execution`, owner-verified like `run_job` since the SDK is not a CI dep; (b) it must be best-effort ? `run_job`'s return falls back to the literal string "execution" when `op.metadata.name` is absent (`cloud_run.py:108-112`), which is not a resolvable execution name; (c) the "check the marker once more before failing" suggestion is good and cheap; also consider aligning `max_wait_sec` with the job's task-timeout. Owner-gating: the shim fix + fake-client tests are doable now; only the real-client cancel verification and any policy retune on the live job are owner-gated (M7 real-run / real cloud spend convention), which matches the finding's own caveat.

- verifier reasoning: Personally reproduced from current code (post-pull, HEAD e410136). (1) `backend/compute/cloud_run.py:173-178` ? `run_infer` blocks on `poll_until(self._read_marker(done_uri), self._policy)` with no exception handling; `backend/infrastructure/execution_policy.py:54` sets `max_wait_sec=1800.0` default and :183-189 raises `PolicyTimeout` at the deadline. (2) `backend/main.py:1064-1070` catches the exception and returns `_failed(f"cloud dispatch error: {e}", True)` ? the local job goes terminal-failed, so the late done-marker/outputs are never ingested. (3) No cancellation exists anywhere: `grep` for `cancel/executions./CancelExecution` across `backend/*.py` hits only the minified UI asset (`backend/ui/assets/index-ukuS9wxa.js`); `GoogleCloudRunJobClient` (`cloud_run.py:69-112`) only implements `run_job`, and the `CloudRunJobClient ABC (:50-66)` has no cancel method. (4) Not already fixed: `git log -15 -- backend/compute/cloud_run.py` shows only formatting (b6214e5), `isort` (f523d3a), an unrelated project-guard fix (688d28f), and the original M6/provenance commits. (5) The mechanism is worse than a rare edge: `get_compute_backend` (`backend/compute/__init__.py:39,76`) passes `DEFAULT_POLICY (1800s)` while the deployed job uses `--task-timeout 3600` (`deploy/cloudrun/README.md:33`, `docs/CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:57`), so any execution legitimately taking 30-60 min is systematically abandoned at 30 min, keeps billing the L4 for up to another 30 min, then its completed result is dropped. `deploy/cloudrun/README.md:60-64` documents the timeout path but only for hung/crashed containers ? the slow-but-healthy case was not considered.

5. user (2026-07-02T04:46:01.240Z)

```

1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6      --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** `done_uri` via `execution_policy.poll_until` + the storage
layer (D16 ? the
8      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10     cold-start is absorbed by the async poll.
11     3. **Read** the marker (it carries `video_id` + the worker's returncode) ? fetch
12     `outputs/<video_id>/report.json` ? `ComputeResult`.
13
14     The dispatched container is forced to run **INLINE** (`compute_target=local_gpu` +
native detector)
15     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17     The Cloud Run trigger is an **injected client** (`CloudRunJobClient`) so the whole
shim is unit-tested
18     with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-imports `google-cloud-
run` and is
19     owner-verified on the M7 real run.
20     """
21
22     from __future__ import annotations
23
24     import json
25     import logging

```

```

26     import os
27     import tempfile
28     import uuid
29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import (
34         DEFAULT_POLICY,
35         ExecutionPolicy,
36         poll_until,
37     )
38     from backend.storage import fetch, get_storage, parse_uri
39
40     LOG = logging.getLogger("compute.cloud_run")
41
42     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
43     # branch.
44     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
45     # runner.)
46     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
47         "deployment.compute_target=local_gpu",
48         "indexing.detector_impl=native",
49     )
50
51     class CloudRunJobClient(ABC):
52         """Triggers a Cloud Run **Job** execution with per-execution container arg
53         overrides.
54
55         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
56         implementation
57         overrides the job's container ``args`` for this execution and starts it."""
58
59         @abstractmethod
60         def run_job(
61             self,
62             job_name: str,
63             argv: List[str],
64             *,
65             region: str,
66             project: Optional[str] = None,
67         ) -> str:
68             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
69             return an
70             execution id/name (for logs). Does NOT block on completion (the caller bucket-
71             polls)."""
72
73         class GoogleCloudRunJobClient(CloudRunJobClient):
74             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
75             run; CI uses
76             a fake, so this code path is never exercised in tests (the SDK isn't a test
77             dependency)."""
78
79             def run_job(
80                 self,
81                 job_name: str,
82                 argv: List[str],
83                 *,
84                 region: str,
85                 project: Optional[str] = None,
86             ) -> str:
87                 # A None/empty project would build a bogus "projects/None/locations/..." path
88                 (job_path

```

```

82         # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
83         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
84         if not project:
85             raise RuntimeError(
86                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
87                 "to resolve the job path; set it on the control plane (see
get_compute_backend)."
88             )
89         try:
90             from google.cloud import run_v2 # type: ignore[attr-defined]
91         except Exception as exc: # pragma: no cover - real SDK not present in CI
92             raise RuntimeError(
93                 "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
94                 "control plane (it is intentionally NOT a CI/test dependency)."
95             ) from exc
96         client = (
97             run_v2.JobsClient()
98         ) # pragma: no cover - exercised only on the real M7 run
99         name = client.job_path(project, region, job_name)
100         overrides = run_v2.RunJobRequest.Overrides(
101             container_overrides=[
102                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
103             ]
104         )
105         op = client.run_job(
106             request=run_v2.RunJobRequest(name=name, overrides=overrides)
107         )
108         return (
109             getattr(op, "metadata", None)
110             and getattr(op.metadata, "name", "")
111             or "execution"
112         )
113
114
115 class CloudRunCompute(ComputeBackend):
116     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
117
118     def __init__(
119         self,
120         *,
121         run_client: CloudRunJobClient,
122         job_name: str,
123         region: str,
124         project: Optional[str] = None,
125         storage_config: Optional[dict] = None,
126         policy: ExecutionPolicy = DEFAULT_POLICY,
127         token: Optional[str] = None,
128         container_overrides: Optional[tuple[str, ...]] = None,
129     ) -> None:
130         self._client = run_client
131         self._job_name = job_name
132         self._region = region
133         self._project = project
134         self._storage_config = storage_config or {}
135         self._policy = policy
136         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
137         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
138         self._token = token
139         self._container_overrides = (

```

```

140         _DEFAULT_CONTAINER_OVERRIDES
141         if container_overrides is None
142         else tuple(container_overrides)
143     )
144
145     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
146         out_root = spec.out_uri.rstrip("/")
147         token = self._token or uuid.uuid4().hex
148         done_uri = f"{out_root}/_dispatch/{token}.done"
149         argv: List[str] = [
150             "infer",
151             "--video-uri",
152             spec.video_uri,
153             "--out-uri",
154             spec.out_uri,
155             "--done-uri",
156             done_uri,
157         ]
158         if spec.segmenter:
159             argv += ["--segmenter", spec.segmenter]
160         for kv in (*spec.config_overrides, *self._container_overrides):
161             argv += ["--set", kv]
162
163         execution = self._client.run_job(
164             self._job_name, argv, region=self._region, project=self._project
165         )
166         LOG.info(
167             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
168             self._job_name,
169             execution,
170             done_uri,
171         )
172
173         marker = poll_until(
174             lambda: self._read_marker(done_uri),
175             self._policy,
176             label="cloud-run-infer",
177             logger=LOG,
178         )
179         video_id = str(marker.get("video_id", ""))
180         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
181         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
182         rc = int(marker.get("returncode", 1))
183         if not video_id:
184             LOG.warning(
185                 "cloud-run: job=%s completion marker present but empty/unreadable (no "
186                 "video_id) ? treating as failure",
187                 self._job_name,
188             )
189             rc = rc or 1
190         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
191         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
192         LOG.info(
193             "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
194         )
195         return ComputeResult(
196             returncode=rc,
197             outputs_uri=outputs_uri,
198             video_id=video_id,
199             report=report,
200             execution=str(execution or ""),
201         )
202

```

```

203         # --- storage-backed completion signal (bucket-poll) ---
204         def _read_marker(self, done_uri: str) -> Optional[dict]:
205             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206             ref = parse_uri(done_uri)
207             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208                 return None
209             return (
210                 self._read_json(done_uri) or {}
211             ) # empty dict still terminates the poll (present == done)
212
213         def _read_json(self, uri: str) -> dict:
214             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
215             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
216             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
217             try:
218                 ref = parse_uri(uri)
219                 dst = os.path.join(
220                     tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
221                 )
222                 local = fetch(uri, dst, config=self._storage_config)
223                 with open(local, "r", encoding="utf-8") as f:
224                     data: Any = json.load(f)
225                 return data if isinstance(data, dict) else {}
226             except Exception:
227                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
228                 return {}
229

```

6. user (2026-07-02T04:46:11.899Z)

```

1         """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3         Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4         - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7         - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10        **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11        waiting (a fake clock advances instantly).
12
13        **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14        transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15        ``execution.policy`` logger to watch the poll trail.
16        """
17
18        from __future__ import annotations
19
20        import logging
21        import threading
22        import time as _time
23        from dataclasses import asdict, dataclass
24        from enum import Enum

```

```

25     from typing import Callable, Optional, TypeVar
26
27     T = TypeVar("T")
28
29     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30     LOG = logging.getLogger("execution.policy")
31
32
33     class WaitMode(str, Enum):
34         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37         ABORT = "abort" # one attempt; on hang/fail, stop immediately
38         BLOCK = "block" # bounded in-process blocking wait (no reschedule)
39
40
41     class PolicyTimeout(TimeoutError):
42         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45     @dataclass(frozen=True)
46     class ExecutionPolicy:
47         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49         mode: WaitMode = WaitMode.WAIT_AND_RESUME
50         poll_every_n_sec: float = 10.0
51         op_timeout_sec: float = (
52             120.0 # HARD per-attempt deadline ? the hang-killer that was missing
53         )
54         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
55         max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
56         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
57         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
58         resumable: bool = True
59
60         def to_dict(self) -> dict:
61             d = asdict(self)
62             d["mode"] = self.mode.value
63             return d
64
65         @classmethod
66         def from_dict(cls, d: dict) -> "ExecutionPolicy":
67             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
68             kw = {k: v for k, v in d.items() if k in known}
69             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                 kw["mode"] = WaitMode(kw["mode"])
71             return cls(**kw)
72
73
74     #: The sane default (also what a job gets if it declares no policy).
75     DEFAULT_POLICY = ExecutionPolicy()
76
77
78     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80         return policy.backoff_base_sec * (2**attempt)
81
82
83     def run_bounded(
84         fn: Callable[[T], T],

```

```

85     policy: ExecutionPolicy = DEFAULT_POLICY,
86     *,
87     label: str = "op",
88     logger: Optional[logging.Logger] = None,
89     sleep: Callable[[float], None] = _time.sleep,
90 ) -> T:
91     """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93     Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94     last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
95     """
96     log = logger or LOG
97     attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
98     last_exc: Optional[BaseException] = None
99     for attempt in range(attempts):
100         box: dict = {}
101         done = threading.Event()
102
103         # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
104         # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
105         def _runner(_box: dict = box, _done: threading.Event = done) -> None:
106             try:
107                 _box["val"] = fn()
108             except (
109                 BaseException
110             ) as e: # relay ANY error (incl. timeouts) to the caller thread
111                 _box["exc"] = e
112             finally:
113                 _done.set()
114
115         threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
116         if done.wait(timeout=policy.op_timeout_sec):
117             if "exc" in box:
118                 last_exc = box["exc"]
119                 log.info(
120                     "%s: attempt %d/%d failed: %s",
121                     label,
122                     attempt + 1,
123                     attempts,
124                     last_exc,
125                 )
126             else:
127                 if attempt:
128                     log.info(
129                         "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
130                     )
131                 return box["val"] # type: ignore[return-value]
132         else:
133             last_exc = PolicyTimeout(
134                 f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
135             )
136             log.warning(
137                 "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
138                 label,
139                 attempt + 1,
140                 attempts,
141                 policy.op_timeout_sec,
142             )
143         if attempt < attempts - 1:
144             wait = _backoff_sec(policy, attempt)

```

```

145         log.debug("%s: backing off %.1fs before retry", label, wait)
146         sleep(wait)
147     assert last_exc is not None
148     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149     raise last_exc
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160     ) -> T:
161         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
162         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
163
164         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
165         """
166         log = logger or LOG
167         start = monotonic()
168         polls = 0
169         log.info(
170             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
171             label,
172             policy.poll_every_n_sec,
173             policy.max_wait_sec,
174         )
175         while True:
176             result = check()
177             if result is not None:
178                 log.info(
179                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
180                 )
181                 return result
182             elapsed = monotonic() - start
183             if elapsed >= policy.max_wait_sec:
184                 log.warning(
185                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
186                 )
187                 raise PolicyTimeout(
188                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
189                 )
190             polls += 1
191             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
192             sleep(policy.poll_every_n_sec)
193

```

7. user (2026-07-02T04:46:12.969Z)

Found 5 files

```

tests\test_startup_checks.py
tests\test_compute_backend.py
tests\test_config_deployment.py
tests\test_billing.py
tests\test_api_cloud_dispatch.py

```

8. user (2026-07-02T04:46:17.987Z)

```

1         """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).

```

[illegible]

```

60         "status": "success",
61     },
62     f,
63 )
64     return "exec-123"
65
66
67     class _NoopRunClient(CloudRunJobClient):
68         """Records the dispatch but does NO filesystem writes ? for tests that fake the
storage layer
69         directly (a gs:// out_uri has no local dir to makedirs)."""
70
71         def __init__(self):
72             self.calls: list[dict] = []
73
74         def run_job(self, job_name, argv, *, region, project=None):
75             self.calls.append(
76                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
77             )
78             return "exec"
79
80
81     # ----- factory (default-
OFF)
82
83
84     def _cfg(compute_target="", storage_backend="local"):
85         return AppConfig.model_validate(
86             {
87                 "deployment": {"compute_target": compute_target},
88                 "storage": {"backend": storage_backend},
89             }
90         )
91
92
93     def test_factory_is_none_for_local_targets():
94         """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
path)."""
95         assert get_compute_backend(_cfg("")) is None
96         assert get_compute_backend(_cfg("local_gpu")) is None
97         assert get_compute_backend(_cfg("cpu_mock")) is None
98
99
100     def test_factory_returns_cloud_run_for_cloud_target():
101         backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
102         assert isinstance(backend, CloudRunCompute)
103
104
105     # ----- real client: missing-project
guard
106
107
108     def test_real_client_rejects_missing_project():
109         """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is
unset ?
110         a None/empty project would build a bogus "projects/None/..." job path (job_path
requires a
111         real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
112         (the SDK is intentionally not a test dependency)."""
113         from backend.compute.cloud_run import GoogleCloudRunJobClient
114
115         client = GoogleCloudRunJobClient()
116         for missing in (None, ""):
117             with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):

```

```

118         client.run_job(
119             "rally-worker", ["infer"], region="asia-south1", project=missing
120         )
121
122
123     # ----- CloudRunCompute dispatch
+ poll
124
125
126     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
127         client = _FakeRunClient(video_id="abc123", segment_count=3)
128         backend = CloudRunCompute(
129             run_client=client,
130             job_name="rally-worker",
131             region="asia-south1",
132             project="proj",
133             storage_config={},
134             policy=_FAST,
135             token="tok",
136         )
137         out = str(tmp_path / "bucket")
138         result = backend.run_infer(
139             ComputeJobSpec(
140                 video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
141             )
142         )
143
144         assert result.returncode == 0
145         assert result.video_id == "abc123"
146         assert result.outputs_uri == f"{out}/outputs/abc123/"
147         assert result.report.get("segment_count") == 3
148         # exactly one Cloud Run Job execution was triggered, in the right region/project
149         assert len(client.calls) == 1
150         assert (
151             client.calls[0]["job"] == "rally-worker"
152             and client.calls[0]["region"] == "asia-south1"
153         )
154
155
156     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
157         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
158         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
159         client = _FakeRunClient()
160         backend = CloudRunCompute(
161             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
162         )
163         backend.run_infer(
164             ComputeJobSpec(
165                 video_uri="gs://b/v.mp4",
166                 out_uri=str(tmp_path / "bk"),
167                 segmenter="wasb_hybrid",
168                 config_overrides=("sport=tennis",),
169             )
170         )
171
172         argv = client.calls[0]["argv"]
173         assert argv[0] == "infer"
174         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
175         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
176         # the container-inline overrides come last (win) and the user override is carried
through
177         joined = " ".join(argv)
178         assert "deployment.compute_target=local_gpu" in joined

```

```

179     assert "indexing.detector_impl=native" in joined
180     assert "sport=tennis" in joined
181     # user override precedes the inline-forcing overrides in --set order
182     assert joined.index("sport=tennis") < joined.index(
183         "deployment.compute_target=local_gpu"
184     )
185
186
187     def test_unique_token_per_call_when_not_injected(tmp_path):
188         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
189         client = _FakeRunClient()
190         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
191         backend.run_infer(
192             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
193         )
194         backend.run_infer(
195             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
196         )
197         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
198         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
199         assert done1 != done2
200
201
202     def test_read_json_passes_dst_for_cloud_backends(monkeypatch):
203         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a None dst raises) ?
_read_json must
204         pass one, else every real cloud marker/report read silently returns {} (local hid
this via its
205         identity passthrough). Mimic a cloud backend that rejects dst=None."""
206         from backend.compute import cloud_run as cr
207
208         captured = {}
209
210         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
211             captured["dst"] = dst
212             if dst is None:
213                 raise ValueError(
214                     "cloud backend requires a dst path"
215                 ) # mirror GCSStorage/S3Storage
216             with open(dst, "w", encoding="utf-8") as f:
217                 f.write({'video_id': "v", "returncode": 0})
218             return dst
219
220         monkeypatch.setattr(cr, "fetch", fake_fetch)
221         backend = cr.CloudRunCompute(run_client=_FakeRunClient(), job_name="j", region="r")
222         data = backend._read_json("gs://b/x.json")
223         assert (
224             captured["dst"] is not None
225         ) # the fix: a real dst is always passed (no silent {})
226         assert data == {"video_id": "v", "returncode": 0}
227
228
229     def test_cloud_run_polls_until_marker_appears(monkeypatch):
230         """The bucket-poll WAIT path (D7 scale-to-zero cold start): the marker is ABSENT for
the first
231         polls then appears ? poll_until must LOOP, not resolve on poll 0. The synchronous
fake elsewhere
232         never exercises this; here exists() is False twice then True."""
233         from backend.compute import cloud_run as cr
234
235         exists_calls = {"n": 0}
236
237         class _FakeStore:
238             def exists(self, ref):

```

```

239         exists_calls["n"] += 1
240         return exists_calls["n"] >= 3 # absent for polls 1-2, present on poll 3
241
242 monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
243
244 def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
245     if dst is None:
246         dst = os.path.join(tempfile.mkdtemp(prefix="cr-poll-"), "m.json")
247         with open(dst, "w", encoding="utf-8") as f:
248             f.write('{ "video_id": "vlate", "returncode": 0, "segment_count": 1 }')
249         return dst
250
251 monkeypatch.setattr(cr, "fetch", fake_fetch)
252 backend = cr.CloudRunCompute(
253     run_client=_NoopRunClient(),
254     job_name="j",
255     region="r",
256     token="t",
257     policy=ExecutionPolicy(
258         poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0
259     ),
260 )
261 result = backend.run_infer(
262     ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
263 )
264 assert result.video_id == "vlate"
265 assert (
266     exists_calls["n"] >= 3
267 ) # >=1 absent poll occurred before the marker appeared (loop ran)
268
269 # ----- worker wiring: dispatch branch +
done-marker
271
272
273 def test_cmd_infer_dispatches_when_cloud_target(tmp_path, monkeypatch):
274     """cmd_infer with compute_target=cloud_run hands off to the ComputeBackend (no in-
275     process run)."""
276     from backend.compute.base import ComputeResult
277     from backend.worker import __main__ as wmain
278
279     seen = {}
280
281     class _FakeBackend:
282         def run_infer(self, spec):
283             seen["spec"] = spec
284             return ComputeResult(
285                 returncode=0, video_id="vidX", outputs_uri="gs://b/outputs/vidX/"
286             )
287
288 monkeypatch.setattr(wmain, "get_compute_backend", lambda config: _FakeBackend())
289 cfg = tmp_path / "c.json"
290 cfg.write_text(
291     '{"deployment": {"profile": "cloud-serving"}, "storage": {"backend": "gcs"}}'
292 )
293 rc = wmain.cmd_infer(
294     wmain.build_parser().parse_args(
295         [
296             "infer",
297             "--video-uri",
298             "gs://b/videos/clip.mp4",
299             "--out-uri",
300             "gs://b",
301             "--config",
302             str(cfg),

```

```

302         "--segmenter",
303         "wasb_hybrid",
304     ]
305 )
306 )
307 assert rc == 0
308 assert seen["spec"].video_uri == "gs://b/videos/clip.mp4"
309 assert seen["spec"].segmenter == "wasb_hybrid"
310
311
312 def test_worker_writes_done_marker_on_success(tmp_path, monkeypatch):
313     """The container side: cmd_infer with --done-uri writes a completion marker
314     (video_id + rc) so a
315     remote dispatcher's bucket-poll terminates. Driven by the deterministic stub + skip-
316     AI."""
317     from backend.worker import __main__ as wmain
318
319     class _OK:
320         passed = True
321         validator_name = "fake"
322         message = "ok"
323
324         def model_dump(self):
325             return {"validator_name": "fake", "passed": True, "message": "ok"}
326
327     monkeypatch.setattr(
328         wmain.validator_registry,
329         "run_all_with_context",
330         lambda path, cfg: ([_OK()], {}),
331     )
332     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
333     monkeypatch.setattr(
334         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
335         lambda p: 30.0,
336     )
337
338     cfg = tmp_path / "c.json"
339     cfg.write_text(
340         json.dumps(
341             {
342                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
343                 "output": {"output_dir": str(tmp_path / "o")},
344             }
345         )
346     )
347     video = tmp_path / "in.mp4"
348     video.write_bytes(b"FAKEVIDEO")
349     out = tmp_path / "bucket"
350     done = str(out / "_dispatch" / "tok.done")
351
352     rc = wmain.cmd_infer(
353         wmain.build_parser().parse_args(
354             [
355                 "infer",
356                 "--video-uri",
357                 str(video),
358                 "--out-uri",
359                 str(out),
360                 "--config",
361                 str(cfg),
362                 "--scratch",
363                 str(tmp_path / "s"),
364                 "--segmenter",
365                 "wasb_hybrid",
366                 "--done-uri",

```

```

365         done,
366     ]
367 )
368 )
369 assert rc == 0
370 marker = json.loads(open(done, encoding="utf-8").read())
371 assert marker["returncode"] == 0 and marker["status"] == "success"
372 assert marker["video_id"] and marker["segment_count"] >= 1
373
374
375 def test_worker_no_done_uri_is_unchanged(tmp_path, monkeypatch):
376     """DEFAULT-OFF: without --done-uri the inline run writes NO marker (today's
behaviour)."""
377     from backend.worker import __main__ as wmain
378
379     class _OK:
380         passed = True
381         validator_name = "fake"
382         message = "ok"
383
384     def model_dump(self):
385         return {"validator_name": "fake", "passed": True, "message": "ok"}
386
387     monkeypatch.setattr(
388         wmain.validator_registry,
389         "run_all_with_context",
390         lambda path, cfg: ([_OK()], {}),
391     )
392     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
393     monkeypatch.setattr(
394         "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
395         lambda p: 30.0,
396     )
397     cfg = tmp_path / "c.json"
398     cfg.write_text(
399         json.dumps(
400             {
401                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
402                 "output": {"output_dir": str(tmp_path / "o")},
403             }
404         )
405     )
406     video = tmp_path / "in.mp4"
407     video.write_bytes(b"FAKEVIDEO")
408     out = tmp_path / "bucket"
409     rc = wmain.cmd_infer(
410         wmain.build_parser().parse_args(
411             [
412                 "infer",
413                 "--video-uri",
414                 str(video),
415                 "--out-uri",
416                 str(out),
417                 "--config",
418                 str(cfg),
419                 "--scratch",
420                 str(tmp_path / "s"),
421                 "--segmenter",
422                 "wasb_hybrid",
423             ]
424         )
425     )
426     assert rc == 0
427     assert not (out / "_dispatch").exists() # no marker dir when --done-uri is omitted
428

```

```

429
430 def test_worker_writes_failure_marker_on_validation_fail(tmp_path, monkeypatch):
431     """The container side: a worker-side FAILURE (validation rc=2) with --done-uri
writes a marker
432     carrying the REAL rc, so a remote dispatcher's bucket-poll terminates FAST (not a
30-min hang)."""
433     from backend.worker import __main__ as wmain
434
435     class _BAD:
436         passed = False
437         validator_name = "blur"
438         message = "blurry"
439
440         def model_dump(self):
441             return {"validator_name": "blur", "passed": False, "message": "blurry"}
442
443     monkeypatch.setattr(
444         wmain.validator_registry,
445         "run_all_with_context",
446         lambda path, cfg: ([_BAD()], {}),
447     )
448     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
449     cfg = tmp_path / "c.json"
450     cfg.write_text(
451         json.dumps(
452             {
453                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
454                 "output": {"output_dir": str(tmp_path / "o")},
455             }
456         )
457     )
458     video = tmp_path / "in.mp4"
459     video.write_bytes(b"FAKEVIDEO")
460     out = tmp_path / "bucket"
461     done = str(out / "_dispatch" / "tok.done")
462
463     rc = wmain.cmd_infer(
464         wmain.build_parser().parse_args(
465             [
466                 "infer",
467                 "--video-uri",
468                 str(video),
469                 "--out-uri",
470                 str(out),
471                 "--config",
472                 str(cfg),
473                 "--scratch",
474                 str(tmp_path / "s"),
475                 "--segmenter",
476                 "wasb_hybrid",
477                 "--done-uri",
478                 done,
479             ]
480         )
481     )
482     assert rc == 2 # validation failure
483     marker = json.loads(open(done, encoding="utf-8").read())
484     assert marker["returncode"] == 2 and marker["status"] == "failed"
485     assert marker[
486         "video_id"
487     ] # carries the id so the dispatcher's poll terminates (rc != 0 ? failed)
488
489
490 def test_worker_failure_writes_no_marker_without_done_uri(tmp_path, monkeypatch):
491     """DEFAULT-OFF: a worker failure with NO --done-uri writes no marker (the unchanged

```

```

local run)."""
492     from backend.worker import __main__ as wmain
493
494     class _BAD:
495         passed = False
496         validator_name = "blur"
497         message = "blurry"
498
499         def model_dump(self):
500             return {"validator_name": "blur", "passed": False, "message": "blurry"}
501
502     monkeypatch.setattr(
503         wmain.validator_registry,
504         "run_all_with_context",
505         lambda path, cfg: ([_BAD()], {}),
506     )
507     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
508     cfg = tmp_path / "c.json"
509     cfg.write_text(
510         json.dumps(
511             {
512                 "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
513                 "output": {"output_dir": str(tmp_path / "o")},
514             }
515         )
516     )
517     video = tmp_path / "in.mp4"
518     video.write_bytes(b"FAKEVIDEO")
519     out = tmp_path / "bucket"
520     rc = wmain.cmd_infer(
521         wmain.build_parser().parse_args(
522             [
523                 "infer",
524                 "--video-uri",
525                 str(video),
526                 "--out-uri",
527                 str(out),
528                 "--config",
529                 str(cfg),
530                 "--scratch",
531                 str(tmp_path / "s"),
532                 "--segmenter",
533                 "wasb_hybrid",
534             ]
535         )
536     )
537     assert rc == 2
538     assert not (out / "_dispatch").exists()
539
540
541     # ----- Dockerfile structural guard
542     (no build)
543
544     def test_dockerfile_has_required_directives():
545         """CI can't build a CUDA image, so guard the Dockerfile's load-bearing directives
546         structurally:
547         the GPU base, ffmpeg + DejaVu fonts (the watermark), the WASB conda env, and the
548         worker entrypoint."""
549         from pathlib import Path
550
551         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
552         assert "FROM nvidia/cuda" in text # GPU base (CUDA)
553         assert "ffmpeg" in text # stitch/decode
554         assert "fonts-dejavu" in text # the watermark filter font

```

```

553     assert "torch==1.11.0+cu113" in text # the verified WASB stack
554     assert "WASB-SBDT" in text # the shuttle detector repo
555     assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
556     assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
557     # The cloud worker reads the gs:// input + writes the gs:// outputs/done-marker
through
558     # backend.storage, so the image MUST carry the gcs backend deps. A bare `pip install
.` boots
559     # but dies on the first gs:// fetch (ModuleNotFoundError: google.cloud) ? a real
Cloud Run
560     # failure the mock-client tests can't see. Lock the extra in structurally.
561     assert ".[storage-gcs]" in text
562
563
564     def test_dockerfile_uses_micromamba_not_anaconda_tos():
565         """P3a: the WASB env is built with micromamba + conda-forge (BSD, no ToS), NOT
Miniconda +
566         `conda tos accept` (the 2000-employee Anaconda Terms-of-Service trap,
PACKAGING_PLAN.md §3).
567         Locked structurally so a revert to the ToS path fails CI; the env path (WASB_PYTHON)
is unchanged."""
568         from pathlib import Path
569
570         text = Path("deploy/cloudrun/Dockerfile").read_text(encoding="utf-8")
571         assert "micromamba" in text # the conda-forge env builder
572         assert "conda-forge" in text # the BSD, ToS-free channel
573         assert (
574             "MAMBA_ROOT_PREFIX=/opt/conda" in text
575         ) # keeps the env at /opt/conda/envs/wasb
576         assert "Miniconda3" not in text # the Anaconda installer is gone
577         assert "conda tos accept" not in text # no ToS acceptance needed (conda-forge)
578

```

9. user (2026-07-02T04:47:02.189Z)

```

deploy\cloudrun\README.md:66:- **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
tests\test_api_cloud_dispatch.py:5:GPU-free + cloud-free via a FAKE CloudRunJobClient that
writes report.json + intervals.csv + the
tests\test_api_cloud_dispatch.py:21:from backend.compute.cloud_run import CloudRunJobClient
tests\test_api_cloud_dispatch.py:39:class _FakeCloudClient(CloudRunJobClient):
backend\compute\__init__.py:64:     from backend.compute.cloud_run import CloudRunCompute,
GoogleCloudRunJobClient
backend\compute\__init__.py:69:     client = run_client if run_client is not None else
GoogleCloudRunJobClient()
tests\test_compute_backend.py:3:All GPU-free + cloud-free: the Cloud Run trigger is a FAKE
`CloudRunJobClient` that simulates the
tests\test_compute_backend.py:16:from backend.compute.cloud_run import CloudRunCompute,
CloudRunJobClient
tests\test_compute_backend.py:24:class _FakeRunClient(CloudRunJobClient):
tests\test_compute_backend.py:67:class _NoopRunClient(CloudRunJobClient):
tests\test_compute_backend.py:109:     """The real GoogleCloudRunJobClient must fail loudly when
GOOGLE_CLOUD_PROJECT is unset ?
tests\test_compute_backend.py:113:     from backend.compute.cloud_run import
GoogleCloudRunJobClient
tests\test_compute_backend.py:115:     client = GoogleCloudRunJobClient()
backend\compute\cloud_run.py:17:The Cloud Run trigger is an **injected client**
(`CloudRunJobClient`) so the whole shim is unit-tested
backend\compute\cloud_run.py:18:with a fake (no GCP). The real `GoogleCloudRunJobClient` lazy-
imports `google-cloud-run` and is
backend\compute\cloud_run.py:50:class CloudRunJobClient(ABC):
backend\compute\cloud_run.py:69:class GoogleCloudRunJobClient(CloudRunJobClient):
backend\compute\cloud_run.py:121:     run_client: CloudRunJobClient,

```

10. user (2026-07-02T04:47:02.301Z)

```

1      """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3      COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4      ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5      ``None``
6      otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
7      identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
8      changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
9
10     """
11
12     from __future__ import annotations
13
14     import os
15     from typing import Any, Optional
16
17     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
18     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
19
20     __all__ = [
21         "ComputeBackend",
22         "ComputeJobSpec",
23         "ComputeResult",
24         "get_compute_backend",
25     ]
26
27     def _storage_dict(config: Any) -> dict:
28         storage = getattr(config, "storage", None)
29         if storage is None:
30             return {}
31         if hasattr(storage, "model_dump"):
32             return storage.model_dump() # type: ignore[no-any-return]
33         return dict(storage) if isinstance(storage, dict) else {}
34
35     def get_compute_backend(
36         config: Any,
37         *,
38         run_client: Optional[Any] = None,
39         policy: ExecutionPolicy = DEFAULT_POLICY,
40         token: Optional[str] = None,
41         target_override: Optional[str] = None,
42     ) -> Optional[ComputeBackend]:
43         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
44         in-process (the default-OFF path).
45
46         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
47         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
48         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
49         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
50
51         ``target_override`` lets a single request force a target (the SPA compute-picker
maps a
52         per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
the
53         server's configured default. ``None`` falls back to
``config.deployment.compute_target``.

```

```

54     """
55     target = (
56         target_override
57         if target_override is not None
58         else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
59     )
60     if target != "cloud_run":
61         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
62
63     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
64     # selected.
65     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
66
67     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
68     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
69     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
70     client = run_client if run_client is not None else GoogleCloudRunJobClient()
71     return CloudRunCompute(
72         run_client=client,
73         job_name=job_name,
74         region=region,
75         project=project,
76         storage_config=_storage_dict(config),
77         policy=policy,
78         token=token,
79     )

```

11. user (2026-07-02T04:47:09.966Z)

```

1     """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
4     the rally
5     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
6     unchanged ? all
7     GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
8     intervals.csv + the
9     done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
10    compute_target=cloud_run
11    dispatches; every other target runs the in-process segmenter byte-identically.
12    """
13
14    import csv
15    import json
16    import os
17
18    import pytest
19
20    pytest.importorskip("httpx")
21    from fastapi.testclient import TestClient
22
23    import backend.compute as compute_pkg
24    import backend.main as m
25    from backend.compute.cloud_run import CloudRunJobClient
26    from backend.config.models import AppConfig
27    from backend.infrastructure.database import Database
28    from backend.infrastructure.execution_policy import ExecutionPolicy
29    from backend.pipeline.validators.base import ValidationResult
30
31    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
32
33    def _pass():
34        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]

```

```

32
33
34 class _InlineExecutor:
35     def submit(self, fn, *a, **k):
36         fn(*a, **k)
37
38
39 class _FakeCloudClient(CloudRunJobClient):
40     """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41     into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43     def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44         self.calls: list[list[str]] = []
45         self.video_id, self.returncode = video_id, returncode
46         self.rows = (
47             rows
48             if rows is not None
49             else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50         )
51
52     def run_job(self, job_name, argv, *, region, project=None):
53         self.calls.append(list(argv))
54         out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55         done_uri = argv[argv.index("--done-uri") + 1]
56         if self.returncode == 0:
57             outputs = os.path.join(out_uri, "outputs", self.video_id)
58             os.makedirs(outputs, exist_ok=True)
59             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                 json.dump(
61                     {
62                         "video_id": self.video_id,
63                         "segment_count": len(self.rows),
64                         "status": "success",
65                     },
66                     f,
67                 )
68             with open(
69                 os.path.join(outputs, "intervals.csv"),
70                 "w",
71                 newline="",
72                 encoding="utf-8",
73             ) as f:
74                 w = csv.writer(f)
75                 w.writerow(["start", "end", "shot_count", "ending_reason"])
76                 for s, e, sc, er in self.rows:
77                     w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79             with open(done_uri, "w", encoding="utf-8") as f:
80                 json.dump(
81                     {
82                         "video_id": self.video_id,
83                         "returncode": self.returncode,
84                         "segment_count": len(self.rows),
85                         "status": "success",
86                     },
87                     f,
88                 )
89             return "exec-1"
90
91
92 @pytest.fixture
93 def cloud_env(tmp_path, monkeypatch):
94     """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the

```

```

bucket read an
95         identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
hash+validate.""
96         db = Database(str(tmp_path / "t.db")) # noqa: F841
97         monkeypatch.setattr(m, "db_instance", db)
98         monkeypatch.setattr(m, "db_instance", db)
99         bucket = tmp_path / "bucket"
100        bucket.mkdir()
101        cfg = AppConfig.model_validate(
102            { # noqa: F841
103                "deployment": {"compute_target": "cloud_run"},
104                "storage": {"backend": "local", "output_root": str(bucket)},
105            }
106        )
107        monkeypatch.setattr(m, "global_config", cfg)
108        monkeypatch.setattr(m, "global_config", cfg)
109        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
110        # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
111        fake_local = tmp_path / "fetched.mp4"
112        fake_local.write_bytes(b"FAKEVIDEOBYTES")
113        monkeypatch.setattr(
114            m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
115        )
116        monkeypatch.setattr(
117            m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
118        )
119        return db, bucket, monkeypatch
120

```

12. user (2026-07-02T04:47:10.523Z)

```

1030        if input_ref.backend == "local":
1031            return _failed(
1032                "cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
1033                "can't be read by the Cloud Run job. Put the video in your bucket first.",
1034                False,
1035            )
1036        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
1037        if not out_root:
1038            return _failed(
1039                "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
1040                "? that's where the Cloud Run job writes the result.",
1041                False,
1042            )
1043
1044        notify("dispatching (cloud_run)")
1045        logger.info(
1046            "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
1047            req.video_path,
1048            out_root,
1049        )
1050        # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
1051        # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
1052        # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
1053        # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
1054        overrides = []
1055        sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
1056        if sk is not None:

```

```

1057         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
1058     spec = ComputeJobSpec(
1059         video_uri=req.video_path,
1060         out_uri=out_root,
1061         segmenter=seg_name,
1062         config_overrides=tuple(overrides),
1063     )
1064     try:
1065         result = compute.run_infer(
1066             spec
1067         ) # dispatch the Cloud Run Job + bucket-poll to completion
1068     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1069         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
1070         return _failed(f"cloud dispatch error: {e}", True)
1071     if result.returncode != 0:
1072         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
1073     if result.video_id and result.video_id != video_id:
1074         # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
1075         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
1076         logger.warning(
1077             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
1078             "and the worker hash; ingesting under the local id.",
1079             result.video_id,
1080             video_id,
1081         )
1082
1083     notify("ingesting (cloud_run)")
1084     storage_dict = (

```

13. assistant (2026-07-02T04:47:59.058Z)

Now let me check the lint/type config and recent commit style, then set up the worktree.

14. user (2026-07-02T04:48:03.208Z)

```

mypy.ini
pyproject.toml
ruff.toml
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)

```

15. user (2026-07-02T04:48:08.658Z)

[mypy]

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

ignore_missing_imports = True

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

explicit_package_bases = True

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set by create_app() / main()).
Route handlers use request.app.state exclusively; helpers take explicit params.
backend.api.job_worker lifted from quarantine (now takes explicit config/db params).

```
[mypy-backend.main]
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a
backend.main, which are typed `config: dict` but receive the AppConfig migration shim
at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine
until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code
to detectors/person_detector.py (which IS type-clean, no quarantine). What remains
here is pre-existing orchestration debt unrelated to that move (implicit-Optional
process_video defaults, the Success|Failure ABC-override mismatch, a Future[...]
annotation) ? ratchet those out in a focused typing pass, not here.

(wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True
```

```
[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
```

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
bury blame history for zero behavior value).

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
warnings (W) + bugbear (B) + isort (I ? consistent import ordering).

```
target-version = "py38"
```

scratch/ = throwaway debugging scripts that may hit real APIs; it is also
excluded from pytest collection (see pytest.ini).

```
extend-exclude = ["scratch"]
```

```
[lint]
extend-select = ["W", "B", "I"]
```

```
[lint.per-file-ignores]
```

Entrypoint scripts deliberately run setup (load_dotenv, logging config,
sys.path) before importing backend modules ? import order is load-bearing.

```
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
```

```
"scripts/run_process_and_stitch.py" = ["E402"]
```

16. assistant (2026-07-02T04:48:13.023Z)

Now I'll create the isolated worktree branched from origin/master.

17. user (2026-07-02T04:48:17.423Z)

```
Preparing worktree (new branch 'fix/cloud-run-poll-timeout-cancel')
branch 'fix/cloud-run-poll-timeout-cancel' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe
details) (#413)
fix/cloud-run-poll-timeout-cancel
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

18. user (2026-07-02T04:48:27.550Z)

```
1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6      --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10     cold-start is absorbed by the async poll.
11     3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14     The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17     The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19     owner-verified on the M7 real run.
20     """
21
22     from __future__ import annotations
23
24     import json
25     import logging
26     import os
27     import tempfile
28     import uuid
29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import (
34         DEFAULT_POLICY,
35         ExecutionPolicy,
36         poll_until,
37     )
```

```

38     from backend.storage import fetch, get_storage, parse_uri
39
40     LOG = logging.getLogger("compute.cloud_run")
41
42     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
43     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
44     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
45         "deployment.compute_target=local_gpu",
46         "indexing.detector_impl=native",
47     )
48
49
50     class CloudRunJobClient(ABC):

```

19. user (2026-07-02T04:48:35.341Z)

```

50     class CloudRunJobClient(ABC):
51         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
52
53         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
54         overrides the job's container ``args`` for this execution and starts it."""
55
56         @abstractmethod
57         def run_job(
58             self,
59             job_name: str,
60             argv: List[str],
61             *,
62             region: str,
63             project: Optional[str] = None,
64         ) -> str:
65             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
66             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
67
68
69         class GoogleCloudRunJobClient(CloudRunJobClient):
70             """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
71             a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
72
73             def run_job(
74                 self,
75                 job_name: str,
76                 argv: List[str],
77                 *,
78                 region: str,
79                 project: Optional[str] = None,
80             ) -> str:
81                 # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
82                 # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
83                 # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
84                 if not project:
85                     raise RuntimeError(
86                         "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "

```

```

87         "to resolve the job path; set it on the control plane (see
get_compute_backend)."
88     )
89     try:
90         from google.cloud import run_v2 # type: ignore[attr-defined]
91     except Exception as exc: # pragma: no cover - real SDK not present in CI
92         raise RuntimeError(
93             "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
94             "control plane (it is intentionally NOT a CI/test dependency)."
95         ) from exc
96     client = (
97         run_v2.JobsClient()
98     ) # pragma: no cover - exercised only on the real M7 run
99     name = client.job_path(project, region, job_name)
100     overrides = run_v2.RunJobRequest.Overrides(
101         container_overrides=[
102             run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
103         ]
104     )
105     op = client.run_job(
106         request=run_v2.RunJobRequest(name=name, overrides=overrides)
107     )
108     return (
109         getattr(op, "metadata", None)
110         and getattr(op.metadata, "name", "")
111         or "execution"
112     )
113
114
115 class CloudRunCompute(ComputeBackend):
116     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
117
118     def __init__(
119         self,
120         *,
121         run_client: CloudRunJobClient,
122         job_name: str,
123         region: str,
124         project: Optional[str] = None,
125         storage_config: Optional[dict] = None,
126         policy: ExecutionPolicy = DEFAULT_POLICY,
127         token: Optional[str] = None,
128         container_overrides: Optional[tuple[str, ...]] = None,
129     ) -> None:
130         self._client = run_client
131         self._job_name = job_name
132         self._region = region
133         self._project = project
134         self._storage_config = storage_config or {}
135         self._policy = policy
136         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
137         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
138         self._token = token
139         self._container_overrides = (
140             _DEFAULT_CONTAINER_OVERRIDES
141             if container_overrides is None
142             else tuple(container_overrides)
143         )
144
145     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
146         out_root = spec.out_uri.rstrip("/")
147         token = self._token or uuid.uuid4().hex

```

```

148     done_uri = f"{out_root}/_dispatch/{token}.done"
149     argv: List[str] = [
150         "infer",
151         "--video-uri",
152         spec.video_uri,
153         "--out-uri",
154         spec.out_uri,
155         "--done-uri",
156         done_uri,
157     ]
158     if spec.segmenter:
159         argv += ["--segmenter", spec.segmenter]
160     for kv in (*spec.config_overrides, *self._container_overrides):
161         argv += ["--set", kv]
162
163     execution = self._client.run_job(
164         self._job_name, argv, region=self._region, project=self._project
165     )
166     LOG.info(
167         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
168         self._job_name,
169         execution,
170         done_uri,
171     )
172
173     marker = poll_until(
174         lambda: self._read_marker(done_uri),
175         self._policy,
176         label="cloud-run-infer",
177         logger=LOG,
178     )
179     video_id = str(marker.get("video_id", ""))
180     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
181     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
182     rc = int(marker.get("returncode", 1))
183     if not video_id:
184         LOG.warning(
185             "cloud-run: job=%s completion marker present but empty/unreadable (no "
186             "video_id) ? treating as failure",
187             self._job_name,
188         )
189     rc = rc or 1
190     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
191     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
192     LOG.info(
193         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
194     )
195     return ComputeResult(
196         returncode=rc,
197         outputs_uri=outputs_uri,
198         video_id=video_id,
199         report=report,
200         execution=str(execution or ""),
201     )
202
203     # --- storage-backed completion signal (bucket-poll) ---
204     def _read_marker(self, done_uri: str) -> Optional[dict]:
205         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
206         ref = parse_uri(done_uri)
207         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
208             return None
209         return (

```

```

210         self._read_json(done_uri) or {}
211     ) # empty dict still terminates the poll (present == done)
212
213     def _read_json(self, uri: str) -> dict:
214         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
215         # raises ValueError ? unlike local's identity passthrough. Without an explicit
216         # would silently return {} for every gs://|s3:// object (the only backends this
217         # shim targets).
218         try:
219             ref = parse_uri(uri)
220             dst = os.path.join(
221                 tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
222             )
223             local = fetch(uri, dst, config=self._storage_config)
224             with open(local, "r", encoding="utf-8") as f:
225                 data: Any = json.load(f)
226             return data if isinstance(data, dict) else {}
227         except Exception:
228             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
229             return {}

```

20. assistant (2026-07-02T04:48:42.903Z)

Now I'll apply the edits to `cloud\_run.py`.

21. user (2026-07-02T04:48:48.059Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

22. user (2026-07-02T04:48:54.101Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. user (2026-07-02T04:49:02.995Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

24. user (2026-07-02T04:49:14.945Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

25. user (2026-07-02T04:49:22.922Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

26. user (2026-07-02T04:49:35.596Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

27. assistant (2026-07-02T04:49:46.564Z)

Now update the test files in the worktree.

28. user (2026-07-02T04:49:47.893Z)

```
1      """ComputeBackend seam ? Cloud Run dispatch shim (COMPUTE_DECOUPLED_SERVING M6).
2
3      All GPU-free + cloud-free: the Cloud Run trigger is a FAKE ``CloudRunJobClient`` that
4      simulates the
5      worker (writes the done-marker + report to a local "bucket" = a tmp dir), so the
6      dispatch?bucket-poll
7      ?read flow is proven for $0. The cardinal invariant: **default-OFF** ? only
8      ``compute_target=cloud_run``
9      selects a backend; everything else returns ``None`` (= run the worker's existing in-
10     process path).
11     """
12
13     import json
14     import os
15     import tempfile
16
17     import pytest
18
19     from backend.compute import ComputeJobSpec, get_compute_backend
20     from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
21     from backend.config import AppConfig
22     from backend.infrastructure.execution_policy import ExecutionPolicy
23
24     # A fast, no-wait policy (the fake client writes the marker synchronously, so the first
25     poll hits).
26     _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
27
28     class _FakeRunClient(CloudRunJobClient):
29         """Simulates a Cloud Run Job: on dispatch, write the report + done-marker into the
30         local bucket
31         exactly where the real worker would, so CloudRunCompute's bucket-poll resolves
32         immediately."""
33
34         def __init__(self, *, video_id="vidCloud", segment_count=2, returncode=0):
35             self.calls: list[dict] = []
36             self.video_id, self.segment_count, self.returncode = (
37                 video_id,
38                 segment_count,
39                 returncode,
40             )
41
42         def run_job(self, job_name, argv, *, region, project=None):
43             self.calls.append(
44                 {"job": job_name, "argv": list(argv), "region": region, "project": project}
45             )
46             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
47             done_uri = argv[argv.index("--done-uri") + 1]
48             outputs = os.path.join(out_uri, "outputs", self.video_id)
49             os.makedirs(outputs, exist_ok=True)
50             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
51                 json.dump(
52                     {
53                         "video_id": self.video_id,
```

```

48         "segment_count": self.segment_count,
49         "status": "success",
50     },
51     f,
52 )
53 os.makedirs(os.path.dirname(done_uri), exist_ok=True)
54 with open(done_uri, "w", encoding="utf-8") as f:
55     json.dump(
56         {
57             "video_id": self.video_id,
58             "returncode": self.returncode,
59             "segment_count": self.segment_count,
60             "status": "success",
61         },
62         f,
63     )
64     return "exec-123"
65
66
67 class _NoopRunClient(CloudRunJobClient):
68     """Records the dispatch but does NO filesystem writes ? for tests that fake the
69     storage layer
70     directly (a gs:// out_uri has no local dir to makedirs)."""
71
72     def __init__(self):
73         self.calls: list[dict] = []
74
75     def run_job(self, job_name, argv, *, region, project=None):
76         self.calls.append(
77             {"job": job_name, "argv": list(argv), "region": region, "project": project}
78         )
79         return "exec"
80
81 # ----- factory (default-
82 OFF)
83
84 def _cfg(compute_target="", storage_backend="local"):
85     return AppConfig.model_validate(
86         {
87             "deployment": {"compute_target": compute_target},
88             "storage": {"backend": storage_backend},
89         }
90     )
91
92
93 def test_factory_is_none_for_local_targets():
94     """DEFAULT-OFF: no/local/cpu_mock compute target ? None (run the in-process
95     path)."""
96     assert get_compute_backend(_cfg("")) is None
97     assert get_compute_backend(_cfg("local_gpu")) is None
98     assert get_compute_backend(_cfg("cpu_mock")) is None
99
100 def test_factory_returns_cloud_run_for_cloud_target():
101     backend = get_compute_backend(_cfg("cloud_run", "gcs"), run_client=_FakeRunClient())
102     assert isinstance(backend, CloudRunCompute)
103
104
105 # ----- real client: missing-project
106 guard
107
108 def test_real_client_rejects_missing_project():

```

```

109     """The real GoogleCloudRunJobClient must fail loudly when GOOGLE_CLOUD_PROJECT is
unset ?
110     a None/empty project would build a bogus "projects/None/..." job path (job_path
requires a
111     real str). The guard runs BEFORE the google-cloud-run import, so it is exercisable
in CI
112     (the SDK is intentionally not a test dependency)."""
113     from backend.compute.cloud_run import GoogleCloudRunJobClient
114
115     client = GoogleCloudRunJobClient()
116     for missing in (None, ""):
117         with pytest.raises(RuntimeError, match="GOOGLE_CLOUD_PROJECT"):
118             client.run_job(
119                 "rally-worker", ["infer"], region="asia-south1", project=missing
120             )
121
122
123     # ----- CloudRunCompute dispatch
+ poll
124
125
126     def test_cloud_run_dispatch_polls_and_returns_result(tmp_path):
127         client = _FakeRunClient(video_id="abc123", segment_count=3)
128         backend = CloudRunCompute(
129             run_client=client,
130             job_name="rally-worker",

```

29. user (2026-07-02T04:49:49.077Z)

```

39     class _FakeCloudClient(CloudRunJobClient):
40         """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41         into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""
42
43         def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44             self.calls: list[list[str]] = []
45             self.video_id, self.returncode = video_id, returncode
46             self.rows = (
47                 rows
48                 if rows is not None
49                 else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50             )
51
52         def run_job(self, job_name, argv, *, region, project=None):
53             self.calls.append(list(argv))
54             out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55             done_uri = argv[argv.index("--done-uri") + 1]
56             if self.returncode == 0:
57                 outputs = os.path.join(out_uri, "outputs", self.video_id)
58                 os.makedirs(outputs, exist_ok=True)
59                 with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60                     json.dump(
61                         {
62                             "video_id": self.video_id,
63                             "segment_count": len(self.rows),
64                             "status": "success",
65                         },
66                         f,
67                     )
68                 with open(
69                     os.path.join(outputs, "intervals.csv"),
70                     "w",
71                     newline="",
72                     encoding="utf-8",

```

```

73         ) as f:
74             w = csv.writer(f)
75             w.writerow(["start", "end", "shot_count", "ending_reason"])
76             for s, e, sc, er in self.rows:
77                 w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79         with open(done_uri, "w", encoding="utf-8") as f:
80             json.dump(
81                 {
82                     "video_id": self.video_id,
83                     "returncode": self.returncode,
84                     "segment_count": len(self.rows),
85                     "status": "success",
86                 },
87                 f,
88             )
89         return "exec-1"
90
91
92     @pytest.fixture
93     def cloud_env(tmp_path, monkeypatch):

```

30. user (2026-07-02T04:49:59.099Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/tests/test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

31. user (2026-07-02T04:50:12.455Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/tests/test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

32. user (2026-07-02T04:50:26.671Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/tests/test_api_cloud_dispatch.py has been updated successfully. (file state is current in your context ? no need to Read it back)

33. assistant (2026-07-02T04:50:45.307Z)

Now add the new tests.

34. user (2026-07-02T04:50:51.008Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/tests/test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

35. user (2026-07-02T04:51:14.446Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/tests/test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

36. assistant (2026-07-02T04:51:20.222Z)

Now run the targeted tests, ruff, and mypy in the worktree.

37. user (2026-07-02T04:51:54.184Z)

```
..... [100%]
===== warnings summary =====
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
42 passed, 4 warnings in 30.23s
```

38. assistant (2026-07-02T04:51:59.546Z)

All 42 tests pass. Now lint and type checks.

39. user (2026-07-02T04:52:10.634Z)

All checks passed!
backend\storage\gcs.py:31: error: Module "google.cloud" has no attribute "storage" [attr-
defined]
Found 1 error in 1 file (checked 1 source file)

40. user (2026-07-02T04:52:25.769Z)

```
.github/workflows/ci.yml:65: # Type gate (mypy.ini at repo root): lenient green baseline with
an explicit
.github/workflows/ci.yml:67: # up. Typed core deps are installed so mypy sees their real
signatures
.github/workflows/ci.yml:78: - name: Install mypy + typed core deps
.github/workflows/ci.yml:79:   run: python -m pip install mypy pydantic fastapi uvicorn
python-dotenv google-genai numpy
.github/workflows/ci.yml:81:   run: mypy backend training
.github/workflows/ci.yml:135: # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
.github/workflows/nightly.yml:73: # Type gate (mypy.ini at repo root) ? mirrors
ci.yml::typecheck. Typed core
.github/workflows/nightly.yml:74: # deps are installed so mypy sees real signatures (without
them
.github/workflows/nightly.yml:85: - name: Install mypy + typed core deps
.github/workflows/nightly.yml:86:   run: python -m pip install mypy pydantic fastapi
uvicorn python-dotenv google-genai numpy
.github/workflows/nightly.yml:88:   run: mypy backend training
.github/workflows/nightly.yml:253: 'One of ruff / leak-scan / mypy / import-smoke /
full-suite (cov>=70) /',
mypy.ini:1:[mypy]
mypy.ini:6:# backend/ is a namespace package (no root __init__.py) ? required so mypy
mypy.ini:15:[mypy-backend.main]
mypy.ini:22:[mypy-backend.worker.__main__]
mypy.ini:31:[mypy-backend.pipeline.segmenters.yolo_hybrid]
mypy.ini:34:[mypy-backend.pipeline.segmenters.gemini]
mypy.ini:38:[mypy-backend.pipeline.segmenters.base]
```

41. assistant (2026-07-02T04:52:36.231Z)

CI runs `mypy backend training` in an env without google-cloud-storage installed, so this looks like a local-env artifact. Let me confirm it's pre-existing (independent of my change).

42. user (2026-07-02T04:52:41.868Z)

```
backend\storage\gcs.py:31: error: Module "google.cloud" has no attribute "storage" [attr-
defined]
Found 1 error in 1 file (checked 1 source file)
M backend/compute/cloud_run.py
M tests/test_api_cloud_dispatch.py
M tests/test_compute_backend.py
```

43. assistant (2026-07-02T04:52:51.183Z)

Pre-existing at base ? confirmed local-env artifact, not introduced by my change. Let me review the final diff before committing.

44. user (2026-07-02T04:52:54.665Z)

```
backend/compute/cloud_run.py      | 91 ++++++-----
tests/test_api_cloud_dispatch.py  | 5 ++
tests/test_compute_backend.py     | 133 ++++++-----
3 files changed, 221 insertions(+), 8 deletions(-)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index ddfc085..5ccaa77 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -7,7 +7,10 @@ Flow (the cloud-serving compute path):
2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
    completion signal is a tiny marker object the worker writes, NOT the Cloud Run API status,
so
    it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero (D7):
the
-    cold-start is absorbed by the async poll.
+    cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can undercut
the
+    job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
healthy
+    worker that finished just past the deadline is rescued, not wasted ? then best-effort
CANCELS
+    the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
    ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -33,6 +36,7 @@ from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
from backend.infrastructure.execution_policy import (
    DEFAULT_POLICY,
    ExecutionPolicy,
+    PolicyTimeout,
    poll_until,
)
from backend.storage import fetch, get_storage, parse_uri
@@ -65,6 +69,13 @@ class CloudRunJobClient(ABC):
    """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return
an
    execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""

+    @abstractmethod
+    def cancel_execution(self, execution: str) -> None:
+        """Cancel a running execution by the name ``run_job`` returned. Called when the bucket-
poll
+        times out so the abandoned execution doesn't keep billing the GPU until ``--task-
timeout``.
+        May raise ? the caller treats every failure as best-effort (the original poll timeout
is
+        NEVER masked by a cancel error)."""
```

```

+
class GoogleCloudRunJobClient(CloudRunJobClient):
    """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI
    uses
@@ -111,6 +122,29 @@ class GoogleCloudRunJobClient(CloudRunJobClient):
        or "execution"
    )

+    def cancel_execution(self, execution: str) -> None:
+        # run_job falls back to the literal string "execution" when the operation metadata
        carries
+        # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK import
+        # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI (where
+        # google-cloud-run is absent).
+        if "/executions/" not in (execution or ""):
+            raise RuntimeError(
+                f"not a fully-qualified Cloud Run execution name: {execution!r} ? expected "
+                "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not resolve
+                "
+                "one from the operation metadata, so there is nothing to cancel by name)."
+            )
+        try:
+            from google.cloud import run_v2 # type: ignore[attr-defined]
+        except Exception as exc: # pragma: no cover - real SDK not present in CI
+            raise RuntimeError(
+                "google-cloud-run is required to cancel a Cloud Run execution; install it on
the "
+                "control plane (it is intentionally NOT a CI/test dependency)."

```

```

+     def _handle_poll_timeout(
+         self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
+     ) -> dict:
+         """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
+
+         The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed job's own
+         `--task-timeout` (3600s), so a slow-but-healthy worker may finish just past the
deadline:
+         ONE final marker check rescues that completed GPU run instead of wasting it. If the
marker
+         is genuinely absent, the execution is still running ? best-effort cancel it (otherwise
the
+         L4 keeps billing until `--task-timeout`) and re-raise a `PolicyTimeout` carrying
the
+         execution name so the operator can verify the state in the GCP console. A cancel
failure
+         NEVER masks the original timeout."""
+         late = self._read_marker(done_uri)
+         if late is not None:
+             LOG.warning(
+                 "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check "
+                 "? rescuing the completed result",
+                 self._job_name,
+                 execution,
+             )
+             return late
+         try:
+             self._client.cancel_execution(execution)
+         except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the timeout
+             cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
+             LOG.warning(
+                 "cloud-run: best-effort cancel of execution=%s failed: %s",
+                 execution,
+                 cancel_exc,
+                 exc_info=True,
+             )
+         else:
+             cancel_note = "cancel requested"
+             LOG.info("cloud-run: requested cancel of execution=%s", execution)
+             raise PolicyTimeout(
+                 f"{timeout_exc} [cloud-run job={self._job_name} "
+                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution "
+                 "state in the GCP console]"
+             ) from timeout_exc
+
+     # --- storage-backed completion signal (bucket-poll) ---
+     def _read_marker(self, done_uri: str) -> Optional[dict]:
+         """The done-marker dict if the worker has written it, else None ? poll_until keeps
waiting."""

```

45. user (2026-07-02T04:53:11.438Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel/backend/compute/cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

46. user (2026-07-02T04:53:13.700Z)

688d28f fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
fix(compute): guard GoogleCloudRunJobClient against an unset GOOGLE_CLOUD_PROJECT (#342)
run_job passed `project` (Optional[str], sourced from the GOOGLE_CLOUD_PROJECT env in get_compute_backend, which can be None) straight to
`JobsClient.job\_path(project: str, ...)` . mypy flags this as an arg-type error once google-cloud-run is installed; CI stays green only because the SDK is

intentionally NOT a CI/test dependency. It is a real latent bug: a None/empty project would build a bogus "projects/None/locations/..." path on the real Cloud Run dispatch.

Validate `project` BEFORE the lazy SDK import and raise a clear RuntimeError naming GOOGLE_CLOUD_PROJECT. This both narrows Optional[str] -> str for job_path (mypy clean) and fails loudly on a misconfigured control plane.

47. user (2026-07-02T04:53:53.012Z)

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
42 passed, 4 warnings in 25.31s
All checks passed!
backend\storage\gcs.py:31: error: Module "google.cloud" has no attribute "storage" [attr-
defined]
Found 1 error in 1 file (checked 1 source file)
```

48. assistant (2026-07-02T04:54:07.640Z)

All green. Now commit with explicit paths after re-verifying the branch state.

49. user (2026-07-02T04:54:11.345Z)

```
fix/cloud-run-poll-timeout-cancel
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
M backend/compute/cloud_run.py
M tests/test_api_cloud_dispatch.py
M tests/test_compute_backend.py
```

50. user (2026-07-02T04:54:24.669Z)

```
[fix/cloud-run-poll-timeout-cancel 1d94e2b] fix(compute): rescue-or-cancel on Cloud Run bucket-
poll timeout (R1-21)
3 files changed, 228 insertions(+), 8 deletions(-)
1d94e2b fix(compute): rescue-or-cancel on Cloud Run bucket-poll timeout (R1-21)
```

51. assistant (2026-07-02T04:54:30.570Z)

Only my commit is on the branch. Now push and open the PR.

52. user (2026-07-02T04:54:35.556Z)

```
remote:
remote: Create a pull request for 'fix/cloud-run-poll-timeout-cancel' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/cloud-run-
poll-timeout-cancel
remote:
branch 'fix/cloud-run-poll-timeout-cancel' set up to track 'origin/fix/cloud-run-poll-timeout-
cancel'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
* [new branch]      fix/cloud-run-poll-timeout-cancel -> fix/cloud-run-poll-timeout-cancel
```

53. assistant (2026-07-02T04:54:40.655Z)

Let me check the PR template before creating the PR.

54. user (2026-07-02T04:54:42.727Z)

```
1      <!-- Describe the change and link the issue it closes. -->
2
3      ## Summary
4
5
```

```
6
7     ## Checks
8     - [ ] `python run_all_tests.py` green; `ruff check .` + `mypy` clean; coverage not
lowered.
9     - [ ] Internal markdown links valid (`python -m tools.check_md_links`).
10
11     ## Launch / "bit-flip" PRs only
12     A PR that flips a default / "launch bit" so a tracked project reaches terminal state
MUST also run
13     the `docs/DOC_STATUS.md` §2 archival due-diligence on that project's docs, **in this
same PR**:
14     - [ ] Verified each doc's claims against the code (tagged load-bearing facts
`[verified]`).
15     - [ ] Flipped the doc Status ? DELIVERED/DONE/REJECTED + a completion note (what
shipped, PR, paths).
16     - [ ] Updated all inbound references (`docs/README.md`, `CLAUDE.md`,
`docs/NEXT_STEPS.md`, cross-doc links).
17     - [ ] Kept the ADR forward/back lineage.
18     - [ ] `git mv` the terminal docs to `docs/archives/?` and recorded the move in
`DOC_STATUS.md` §3.
19     - [ ] Fixed any internal-link breakage from the move (`link-check` CI job green).
20
21     _(Not a launch/bit-flip PR? Delete this section.)_
22
```

55. user (2026-07-02T04:55:12.917Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/417>

56. assistant (2026-07-02T04:55:19.860Z)

PR created. Summarizing the work.

57. user (2026-07-02T04:55:36.138Z)

Structured output provided successfully